PROGRAMAÇÃO ORIENTADA A OBJETOS

Sistema de Oficina Mecânica

Projeto prático — Unidade 4, Aula 16

RELATÓRIO DE IMPLEMENTAÇÃO

Aluno	Kayky Moreira Assunção
Matrícula	406392088938
Curso	Ciência da Computação — 6º período noturno
Disciplina	Programação Orientada a Objetos
Professor	Cleidson Daniel
Data	28 de outubro de 2026

1. O que foi solicitado

A atividade propõe o desenvolvimento de um sistema simples de oficina mecânica em Python, executado pelo terminal e estruturado com os principais conceitos de Programação Orientada a Objetos.

- Cadastrar veículos com placa, modelo e ano.
- Abrir ordens de serviço associando cliente e veículo.
- Registrar serviços de manutenção, como troca de óleo e alinhamento.
- Calcular automaticamente o valor total de cada ordem.
- Listar ordens e exibir o orçamento detalhado.
- Validar entradas e apresentar mensagens amigáveis em caso de erro.
- Disponibilizar um menu interativo e um README com instruções de execução.

Entregáveis preparados

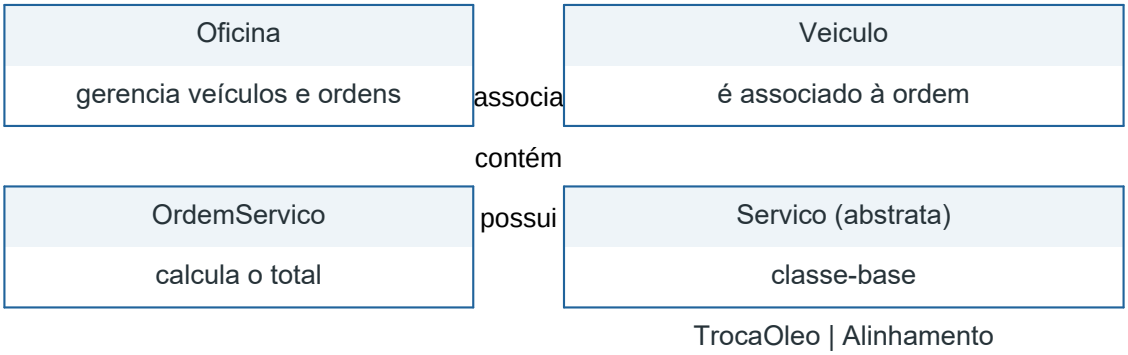
Item	Conteúdo
Código-fonte	Classes separadas, menu interativo e modo de demonstração.
Documentação	README, explicação da solução e diagrama de classes.
Qualidade	20 testes automatizados e evidências da execução.
Relatório	Este PDF com requisitos, arquitetura, resultados e instruções.

O sistema utiliza apenas a biblioteca padrão do Python e, portanto, não exige instalação de dependências externas.

2. Arquitetura e conceitos de POO

Classe	Responsabilidade	Conceito aplicado
Veiculo	Armazena e valida placa, modelo e ano.	Encapsulamento e properties
Servico	Define o contrato comum de cálculo.	Classe abstrata
TrocaOleo	Soma mão de obra e materiais.	Herança e polimorfismo
Alinhamento	Soma mão de obra e taxa do equipamento.	Herança e polimorfismo
OrdemServico	Agrupa cliente, veículo e serviços.	Associação e composição
Oficina	Gerencia cadastros, buscas e ordens.	Agregação e fachada

Relacionamento entre as classes



A lista de serviços contém objetos diferentes que respondem ao mesmo método calcular_valor(). Isso é polimorfismo: a ordem soma os valores sem precisar conhecer a fórmula interna de cada serviço.

3. Implementação

Encapsulamento e validação

Os atributos importantes são protegidos e acessados por propriedades. Ao atribuir a placa, o modelo ou o ano, a própria classe valida o valor. A placa aceita tanto o padrão antigo (ABC1234) quanto o Mercosul (ABC1D23), e o ano deve ficar entre 1886 e 2100.

Classe abstrata e serviços concretos

A classe Servico não pode ser instanciada diretamente. Ela declara o método abstrato calcular_valor(). A troca de óleo calcula mão de obra + materiais; o alinhamento calcula mão de obra + taxa de equipamento.

Ordem e orçamento

A ordem guarda um veículo e uma coleção de serviços. O método calcular_total() percorre a coleção e soma o preço polimórfico de cada item. O método resumo() produz o orçamento detalhado para exibição.

Tratamento de erros

As exceções DadosInvalidosError e ItemNaoEncontradoError diferenciam dados inválidos de buscas sem resultado. O menu captura essas exceções e mantém o programa em execução, apresentando uma mensagem clara ao usuário.

Estrutura de arquivos

```
main.py veiculo.py servico.py ordem_servico.py oficina.py exceptions.py tests/test_sistema.py docs/
evidencias/
```

4. Funcionamento e resultados

Fluxo de uso

- Cadastrar um veículo no menu, informando placa, modelo e ano.
- Abrir uma ordem para a placa cadastrada e informar o cliente.
- Lançar um ou mais serviços na ordem.
- Listar as ordens ou solicitar o orçamento detalhado.

Demonstracão validada

```
ORDEM DE SERVIÇO Nº 1
Cliente: Kayky Moreira Assunção
Veículo: Honda Civic (2022) — placa ABC1D23
Serviços:
1. Troca de óleo: R$ 250,00
2. Alinhamento: R$ 150,00
TOTAL: R$ 400,00
```

Testes automatizados

Foram executados 20 testes cobrindo veículos, serviços, ordens, oficina, cálculos, encapsulamento e erros. Todos terminaram com resultado OK.

Área	Verificações principais	Resultado
Veículo	Placas, modelo e intervalo do ano	Aprovado
Serviços	Abstração, fórmulas e valores inválidos	Aprovado
Ordem	Composição, resumo e total	Aprovado
Oficina	Cadastro, duplicidade, busca e numeração	Aprovado

5. Como executar e conferir

Abra a pasta Projeto Oficina Mecanica no VS Code e use o terminal integrado.

Executar o menu interativo

```
python main.py
```

Executar a demonstracão automática

```
python main.py --demo
```

Executar os testes

```
python -m unittest discover -s tests -v
```

Conteúdo recomendado para envio

- Arquivo ZIP da entrega, contendo o projeto completo e este relatório.
- Link do repositório no GitHub, caso a plataforma possua campo de comentário.
- Não é necessário enviar separadamente arquivos que já estão dentro do ZIP.

Repositório

github.com/Kayky04Moreira/kayky/tree/main/atividades/atividade-10-oficina-mecanica

Conclusão

Os requisitos apresentados na Aula 16 foram implementados: cadastro, ordens, serviços, orçamento, listagem, validações, exceções e menu. A solução demonstra encapsulamento, abstração, herança, polimorfismo e associação, com documentação e testes para comprovar o funcionamento.

6. Identificação e checklist final

Aluno

• Kayky Moreira Assunção — matrícula 406392088938

Checklist de atendimento

Requisito	Situação
Cadastro de veículos	Concluído
Registro de serviços	Concluído
Cálculo da ordem	Concluído
Listagem e orçamento	Concluído
Validações e tratamento de erros	Concluído
Menu interativo	Concluído
README e organização do projeto	Concluído
Testes e evidências	20/20 aprovados

Entrega preparada e validada em 28 de outubro de 2026.